

AgentPay Guard — Submission Document

Project title

AgentPay Guard — policy preflight for programmable agent payments

Short description

AgentPay Guard is a preflight policy and audit layer for AI-agent payment intents before x402 / Circle Gateway / Arc-style payment flows. It evaluates whether an autonomous payment should proceed, returns `ALLOW`, `REVIEW`, or `BLOCK`, and writes a deterministic JSONL audit record.

Track submitted for

Best Agentic Economy Experience on Arc

Circle Developer Account email

kirillnedoboy@gmail.com

Circle products used on Arc

- USDC — target settlement rail for stablecoin-denominated agent payments.
- Circle Gateway — target routing / treasury movement layer after an intent is approved.
- Nanopayments — target fit for high-frequency agent/API/content payments.
- x402-style paid API flows — used as the payment-intent context the Guard checks before execution.

Current MVP note: AgentPay Guard does not move real funds. It is the policy and audit layer before payment execution.

Problem

AI agents and machine clients can initiate stablecoin payments for APIs, data, compute, telemetry, and services faster than a human can review each spend.

Before an autonomous payment executes, builders need deterministic answers:

- Is this agent allowed to pay?
- Is the recipient trusted?
- Is the amount within limits?
- Is the scenario allowed?

- Should this be allowed, reviewed, or blocked?
- Is there an audit trail for the decision?

Without this layer, autonomous payments are harder to explain, test, and operate.

Solution

AgentPay Guard evaluates a payment intent before payment execution.

It validates required fields, applies deterministic policy rules, computes a risk score, returns `ALLOW`, `REVIEW`, or `BLOCK`, and writes a JSONL audit record.

The MVP includes:

- local demo UI;
- `POST /api/payment-intents/evaluate`;
- `GET /api/audit-log`;
- three verified scenarios;
- deterministic policy engine;
- decimal-string money handling;
- audit log and idempotency support;
- tests, lint, typecheck, and production build verification.

Architecture

```text

AI Agent

- > AgentPay Guard
  - > x402 / Circle Gateway / Arc payment flow
  - > Paid API / Service
- ```

Internal MVP flow:

```text

Payment intent

- > request validation
- > policy config
- > deterministic policy engine
- > risk score + decision
- > JSONL audit log
- > demo UI result

```

## Demo scenarios

1. API nanopayment: returns `ALLOW`.
2. Machine-to-machine telemetry payment: returns `REVIEW`.
3. Risky autonomous spend: returns `BLOCK`.

## Functional MVP evidence

Repository:

<https://github.com/KirillNedoboy/agentpay-guard>

Verified commit:

`9810df8b932ee860af053a43a4ae72f19fac50ae`

Demo video artifact:

`docs/agentpay-guard-demo.mp4`

Demo execution log:

`docs/demo-execution-log.txt`

Screenshots:

- `screenshots/01-allow-decision.png`
- `screenshots/02-review-decision.png`
- `screenshots/03-block-decision.png`
- `screenshots/04-audit-log.png`

## Verification run

The following checks passed on the submitted repository state:

```bash

pnpm test

pnpm lint

pnpm typecheck

pnpm build

```

Live local service proof:

- `agentpay-guard.service`: active

- `GET /`: HTTP 200

- `GET /api/audit-log`: HTTP 200

- `POST /api/payment-intents/evaluate`: HTTP 200 for ALLOW / REVIEW / BLOCK scenarios

Test summary:

- 3 test files passed
- 19 tests passed

## **Circle Product Feedback**

### **Why these products**

The project is built around the agentic-payment moment before an AI agent executes a USDC payment through Arc / Circle Gateway / x402-style rails. USDC, Gateway, and Nanopayments are the natural fit because the main use case is small, repeatable, machine-initiated payment intent evaluation.

### **What worked well**

- The Arc / Circle direction is clear for programmable stablecoin commerce.
- The challenge tracks map well to real developer needs, especially agentic payments and nanopayments.
- The Circle docs make it easy to reason about where a preflight control layer should sit before payment execution.

### **What could be improved**

- More end-to-end sample apps for agentic commerce would help builders connect policy, wallet, payment authorization, and settlement.
- A minimal x402 / Gateway buyer-side reference flow would make it easier to demonstrate preflight guardrails before payment.
- More local testnet examples for failed / reviewed / blocked payment attempts would help developers design safer autonomous systems.

### **Recommendations**

- Provide a reference “agent payment authorization” pattern with preflight policy checks.
- Include sample webhook schemas for payment-intent review queues.
- Add examples for budget caps, recipient allowlists, idempotency, and audit logs around Circle payment flows.

### **Explicit limitations**

The MVP is not:

- a payment rail;
- a wallet;
- a custody product;

- AML/KYC software;
- a fraud prevention guarantee;
- an official Arc/Circle module;
- a real-money payment executor.

It does not include private keys, wallet signing, auth, database, smart contracts, or live real-money execution in the current proof.

## **Roadmap**

1. Real x402 / Circle Gateway buyer-side adapter.
2. Policy dashboard for agent spending controls.
3. Operator review queue for `REVIEW` decisions.
4. Webhook examples for agent frameworks.
5. Exportable audit reports.
6. Optional audit hash anchoring.